

1. Context

Velocitor's backend aggregates several pickup and delivery orders that fall on the same customer site into a single job before calling the Route Optimization API, summing the service time and quantities by hand. They do this because the API refuses the natural expression of the same thing: a job may carry a **pickup** or a **delivery** amount, never both. A driver who drops off 6 crates and collects 4 empties at one dock has no way to say so in one job.

The requirement asks us to enhance the Job object so that both quantities can live on one job, while the engine keeps treating the visit as a single stop: one arrival, one service block, one time window.

RouteQ solves the same operational problem with a feature they call **multi-orders** — several orders delivered to the same address are combined into one stop, but *only where doing so improves the plan*, with a defined rule for how their handover times add up. This document adopts their model, their vocabulary, and their metric-driven decision, because it is the shape our customers and our competitors already recognize.

1.1 The optimization engine already supports a job with both quantities

The first thing to establish is where the limitation actually lives, because it is **not** in the solver. The engine models a job as carrying two independent quantity vectors, one for goods dropped off and one for goods collected, and it keeps them separate throughout planning. It never nets one against the other: the delivered quantity has to fit in the vehicle from the depot up to the stop, and the collected quantity has to fit from the stop to the end of the route. Those are two distinct capacity checks and both already run today.

We confirmed the behavior end-to-end against the current staging build. A single job asking to deliver 6 units and collect 4, offered to a vehicle of capacity 10, is assigned: the vehicle leaves the depot carrying 6, and carries 4 after the stop. The same job offered to a vehicle of capacity 5 is correctly rejected for insufficient capacity — even though the *net* change at the stop is only +2, the engine knows six units must physically be on board when the vehicle arrives.

So the literal requirement costs no solver work. It is a request-validation change in MVRO plus a documentation correction. That is section §2.1, and it should ship first and on its own.

1.2 Why that is not enough, and what multi-orders add

Lifting the validation lets Velocitor keep aggregating on their backend. But the aggregation they are forced to perform is lossy and, in one case, silently wrong:

1. **Netting destroys the capacity requirement.** A backend that cannot express both quantities has to collapse "deliver 6, collect 4" into a net delivery of 2. The optimizer then believes a 3-unit van can serve the stop. It cannot — six units have to be on board when it arrives. This is a correctness problem, not a modelling inconvenience, and it exists in every customer integration that merges orders today.
2. **Order identity disappears.** Once several orders become one customer-invented job id, the response comes back under that invented id. Everything downstream of the

API — proof of delivery, billing, the driver app — has to keep a private mapping to recover the real orders.

3. **A backend cannot know whether merging is a good idea.** This is the decisive one. Merging four orders at one address is right when they are small and wrong when their combined load needs a vehicle that does not exist — and the customer's backend, which merges before the request is sent, has no way to tell the difference. Only the optimizer knows what the fleet can absorb. RouteQ's entire design rests on this point: they merge "only if it improves the solution metrics."

This document therefore specifies two things: the **mixed pickup + delivery job** (§2.1), and opt-in **multi-order merging** (§2.2 onward), in which the optimizer decides which same-location orders to combine, reports each combination as one step under a new multi-order id, and hands back the original job ids on that step so MVRO can fan the result out again. Multi-orders are also how the "future scope" line of the requirement — *multiple pickup and delivery operations within a job* — becomes expressible: a merged stop naturally carries both a pickup and a delivery quantity, which §1.1 shows the engine already handles.

Deviation from the requirement, stated up front. The ticket lists automatic merging of jobs, multi-order grouping, and service/quantity aggregation as **out of scope**, to be handled by the customer's backend. §2.1 respects that literally and is sufficient for Velocitor. §2.2 onward deliberately goes further, because point 3 above is not something a backend can solve, because RouteQ already ships it and we are asked about the gap, and because the API-side alternative — accepting pre-merged jobs and hoping they fit — is the status quo that produces unassigned orders no one can explain.

Be clear-eyed about the cost: making the merge decision metric-driven means it is **not** a preprocessing step. The optimizer has to be able to form and dissolve multi-orders while planning and compare the results (§3.1). That is a larger change than merging everything up front, and it is the reason this feature is scoped as its own project rather than as a rider on §2.1. Multi-order merging is **off by default**, so no existing integration, Velocitor's included, changes behavior until it opts in.

2. API Changes

Changes

Scope: Route Optimization **Flexible API only**. Fast API requests using the multi-order options return **400 Bad Request**.

2.1 `jobs[ ].pickup` and `jobs[ ].delivery` on the same job

The rule "a job may specify pickup or delivery, not both" is removed from MVRO request validation and from the API reference. Semantics of a job carrying both:

Aspect	Behavior
--------	----------

Capacity at the stop	The <code>delivery</code> quantity must fit in the vehicle from the route start up to this stop; the <code>pickup</code> quantity must fit from this stop to the route end. Two separate checks — the two are never netted against each other.
<code>load</code> in the response	Post-stop load, i.e. <code>load_before - delivery + pickup</code> .
Arrival, service, time window	Unchanged. One arrival, one setup + service block, one set of <code>time_windows</code> .
Route totals	The job contributes to both <code>routes[].delivery</code> and <code>routes[].pickup</code> .

Three combinations remain rejected, because each is already meaningless or ambiguous with two quantities present:

Condition	Reason
<code>follow_lifo_order</code> with both quantities	LIFO ordering is defined between a load and its matching unload; a stop that does both has no position in that order.
<code>depot_ids</code> with both quantities	A job-to-depot binding says where the goods come from or go to; with both directions present the binding is ambiguous.
<code>split.can_be_split</code> with both quantities	Splitting divides one quantity across vehicles; with two directions there is no single quantity to divide.

2.2 Multi-order options

All request-level settings for this feature live in a new `options.multioorders` object, beside the existing `options.grouping` and `options.objective` objects. Grouping the settings under one object keeps the feature's surface self-contained and leaves room for the follow-ups in §Out of scope without adding more top-level keys. Because the object name already says "multioorder", the field names inside it drop the redundant prefix — the same convention `options.grouping.proximity_factor` already follows.

Parameter	Type	Default	v1 support	Remark
-----------	------	---------	------------	--------

<code>options.multiorders.merge</code>	boolean	<code>false</code>	Supported	Enables multi-order merging. The optimizer combines co-located orders only where the combination improves the plan (§2.3).
<code>options.multiorders.force_merge</code>	boolean	<code>false</code>	Supported	Combines every eligible group unconditionally, whether or not it improves the plan. Requires <code>multiorders.merge</code> . See the warning in §2.3.
<code>options.multiorders.radius_m</code>	number	<code>0</code>	Not supported — must be 0	Proximity radius for treating nearby orders as one address. See the callout below.
<code>jobs[].can_be_merged</code>	boolean	<code>true</code>	Supported	Per-order opt-out. Set <code>false</code> to keep an order out of every multi-order. This one stays on the job object, and keeps RouteQ's name and default.

The semantics and defaults are RouteQ's; only the nesting is ours. For anyone comparing the two APIs: `multiorders.merge` is RouteQ's `merge_multiorders`, `multiorders.force_merge` is `force_merge_multiorders`, and `multiorders.radius_m` is `multiorder_radius_m`.

```
{
  "options": {
    "multiorders": {
      "merge": true,
      "radius_m": 0
    }
  },
  "jobs": [
    { "id": 117, "location_index": 12, "service": 300, "delivery": [6] },
    { "id": 118, "location_index": 12, "service": 300, "pickup": [4] },
  ]
}
```

```
{ "id": 205, "location_index": 12, "service": 300, "delivery": [2], "can_be_merged": false }
]
}
```

The parameter is accepted so that the request shape is stable, but 0 is the only permitted value, and 0 is the default. Any positive value returns 400.

RouteQ's equivalent, `multiover_order_radius_m`, merges orders that fall within that radius of each other, defaulting to 1 m — effectively "the same coordinates, allowing for rounding". Our v1 has **no proximity search at all**: two orders are candidates for merging only when they carry the **same location_index**. Two entrances of one warehouse, or the same building registered twice with slightly different coordinates, will *not* be considered for merging.

The practical consequence for the solution team: a customer whose addresses are geocoded independently per order will see fewer multi-orders than they expect. The workaround until radius support lands is for the customer to deduplicate addresses on their side so that co-located orders share one location entry. Radius-based proximity is the first item of §Out of scope.

2.3 When merging happens: the plan decides

Two separate questions govern every multi-order, and keeping them apart is the key to reading the rest of this document:

- **May these orders merge?** Answered by the eligibility rules of §2.4 — same location, compatible skills, overlapping time windows, and so on. These are hard admissibility rules, evaluated on the request. They produce *candidate* multi-orders.
- **Should these orders merge?** Answered by the optimizer, per candidate, against the plan it is building. A candidate is realized as a multi-order only where doing so leaves the solution better than leaving its orders separate.

"Better" means the optimizer's own total objective — the same figure it minimizes for everything else, combining unassigned-order penalties, travel cost, and soft-constraint penalties. Two consequences follow, and both are worth telling customers:

- **Merging never causes an order to go unassigned that would otherwise have been served.** An unassigned order carries a large penalty, so a combination that no vehicle can absorb loses the comparison decisively. This is the failure mode of backend-side pre-merging (§1.2), and it is structurally impossible here.
- **Merging happens where it pays.** Four small parcels for one office building become one stop with one arrival and one round of parking time. Four pallets for the same building stay separate if that is what lets the fleet carry them.

For the comparison to be honest, per-order costs must be counted on the original orders rather than on the merged stop — otherwise merging would look cheaper simply by reducing the order count, and the optimizer would over-merge. §2.5 covers this.

`multiover_order_force_merge` suppresses the decision and combines every eligible candidate. RouteQ offers the same escape hatch and warns that it "may worsen the planning results", advising it "only if the orders delivered to the same address are significantly smaller than a vehicle's capacity." Carry that warning to customers verbatim.

The legitimate use is operational rather than mathematical: a dispatcher who needs the guarantee that one address is visited exactly once — for gate access, a single signature, or a customer-facing promise — and accepts a worse plan to get it.

2.4 Which co-located jobs are eligible to merge

Sharing a `location_index` is necessary but not sufficient. RouteQ describes their grouping as considering "coordinates, order time windows, and links between orders (the grouping and pickup & delivery pairs)"; our eligibility rules make the equivalent checks explicit. Orders form a candidate multi-order only when every rule below holds. Orders that fail a rule are never candidates and are planned exactly as they are today.

Eligibility **partitions** the jobs at a location into candidate groups plus any number of ineligible jobs — it never fails the request.

Field	Rule	Why
<code>location_index</code>	must be equal	The candidate key in v1 (§2.2).
<code>can_be_merged</code>	must be <code>true</code> on every job	The per-order opt-out.
<code>skills</code>	must be identical	A union would silently raise the bar for orders that never asked for the extra skill.
<code>zones</code>	must be identical	Zone eligibility is evaluated per job against the vehicle's allowed and restricted lists. Neither merging nor intersecting the lists preserves each order's own eligibility, so require them to match.
<code>time_windows</code>	intersection must be non-empty	An empty intersection means the orders genuinely cannot share one arrival. This is RouteQ's "order time windows" criterion.
<code>load_types</code> / <code>incompatible_load_types</code>	the merged sets must not conflict	RouteQ's "order compatibility by tags". A conflict would make the merged stop unservable by construction.

<code>sequence_order</code>	must be equal, or absent on both	Two different positions in a forced sequence cannot be one stop.
<code>joint_order</code>	must be equal, or absent on both	Merging across joint-order groups would change their all-or-nothing semantics.
<code>depot_ids</code>	must be identical, and the merge must not produce both quantities	Follows the rule in §2.1.
<code>split.can_be_split</code>	must be <code>false</code>	Splitting a multi-order would divide several customers' orders as one lot.
<code>follow_lifo_order</code>	must be <code>false</code>	LIFO ordering is defined per order and cannot survive the merge.
<code>volume</code> (3D packing)	must be absent	Volumes are packed as individual items; adding bounding boxes together is not a valid packing.
<code>max_time_in_vehicle</code>	must be absent	A per-job in-vehicle limit has no meaning once several orders share one visit.

Note what is **not** an eligibility rule: vehicle capacity. RouteQ treats capacity as a merge criterion, and so do we — but as part of the *decision* in §2.3, not as an admissibility rule, because whether a combined load fits depends on which vehicle ends up serving it and what else is on that route. Capacity is exactly the judgement the optimizer is better placed to make than a static rule.

Shipment steps are never candidates in v1 — the conservative reading of RouteQ's "links between orders (pickup & delivery pairs)" criterion.

2.5 How multi-order values are calculated

The handover-time model follows RouteQ's directly. RouteQ splits handover into a per-order part that is **summed** (`service_duration_s`) and a per-address part that is taken at its **maximum** (`shared_service_duration_s`, for parking and paperwork). Our existing fields already carry that split: `service` is per-order work, and `setup` is documented as one-time preparation paid once per location. So the mapping is exact and needs no new fields.

Field	Rule	RouteQ equivalent
-------	------	-------------------

<code>service</code>	sum	<code>service_duration_s</code> , summed over the multi-order
<code>setup</code>	max	<code>shared_service_duration_s</code> , maximum over the multi-order
<code>delivery</code>	dimension-wise sum	—
<code>pickup</code>	dimension-wise sum	Kept as its own quantity, never netted against delivery
<code>time_windows</code>	intersection	Time windows are a grouping criterion in RouteQ too
<code>priority</code>	max	Priority is a rank, not a quantity. Summing would let three low-priority orders outrank one urgent one.
<code>revenue</code> , <code>outsourcing_cost</code>	sum	Both are money attached to individual orders
<code>max_visit_lateness</code>	min	The tightest promise wins
<code>description</code>	joined, " ; " -separated	Diagnostics only

Total time at a multi-order stop is therefore **`sum(service) + max(setup)`**, which is exactly RouteQ's stated formula. The saving a merge produces is the travel between the orders (zero, since they share a location), the repeated setup, and one arrival's worth of route overhead — set against a larger combined load on one vehicle.

Per-order costs stay attached to the orders. Two terms in the objective are charged per order rather than per stop: the vehicle's `costs.per_order`, and the penalty applied for each unassigned task. Both are counted on the original orders, so a multi-order of three costs three orders' worth and an unassigned multi-order of three costs three orders' worth of penalty. Without this, merging would appear to reduce cost simply by reducing the number of things being counted, and the decision in §2.3 would be biased toward merging everything — which is the behavior we are specifically trying not to have. `min_stop_load` penalties, by contrast, are genuinely per-stop and correctly see the merged load.

2.6 Response changes (additive)

A realized multi-order produces **one** `job`-type step. RouteQ's documentation does not specify a response representation for multi-orders, so this contract is ours to define.

Field	Type	Remark
-------	------	--------

<code>routes[].steps[].id</code>	integer	The multi-order id — a synthetic id that does not appear anywhere in the request. See §2.7.
<code>routes[].steps[].original_job_ids</code>	array	New. The ids of the merged jobs, ascending, exactly as submitted (string ids stay strings). Present only on multi-order steps.
<code>routes[].steps[].service</code>	(existing)	<code>sum(service)</code> over the merged orders.
<code>routes[].steps[].setup</code>	(existing)	<code>max(setup)</code> over the merged orders.
<code>routes[].steps[].delivery / pickup</code>	(existing fields, wider use)	Today set only for split jobs; now also set on a multi-order step, carrying the summed delivery and pickup quantities so a consumer can see both sides of a mixed stop without re-deriving them.
<code>routes[].steps[].load</code>	(existing)	Post-stop load, <code>load_before - delivery + pickup</code> .
<code>unassigned[]</code>	(existing)	Orders are reported individually under their own ids, exactly as today. A multi-order id never appears in <code>unassigned[]</code> .
<code>summary counters</code>	(existing)	Count original orders, not multi-order stops.

An order that was never merged produces an ordinary step under its own id, with no `original_job_ids` — so a response mixes plain steps and multi-order steps freely, and the presence of the field is the only signal a consumer needs.

```
{
  "steps": [
    {
      "type": "job",
      "id": 4611686018427387904,
```

```

    "original_job_ids": [117, 118],
    "arrival": 34200,
    "setup": 120,
    "service": 600,
    "delivery": [6],
    "pickup": [4],
    "load": [4]
  }
]
}

```

Naming note. The field is `original_job_ids`, the snake_case rendering of the `originalJobIds` used in the requirement discussion. Every other field in the step object is snake_case, and consistency matters more than the spelling in the ticket.

2.7 Multi-order ids

The merged step needs an id, and every choice has a cost. Reusing the smallest original id — the pattern our one-to-many shipment feature uses for its group representative — would make order 118 look dropped to any consumer that does not read `original_job_ids`. A brand-new id makes the "this is a merged stop" signal impossible to miss, at the price of the response containing an id the request never mentioned. This design takes the new id, and §4 is the contract that makes it safe for MVRO.

The rules that matter to a consumer:

1. Multi-order ids come from a **reserved high numeric range** that no submitted id may use, so they can never collide with a customer's own ids. Submitting a job id inside the reserved range is a request error.
2. Ids are assigned to the multi-orders **present in the final plan**, in a fixed order, so the same request solved with the same seed produces the same ids.
3. **Multi-order ids are not durable keys.** Because merging is a planning decision (§2.3), the same address can be one multi-order in today's plan and two ordinary stops in tomorrow's. Consumers must key off `original_job_ids`, never off the multi-order id. §4.3 makes this an integration rule.
4. A multi-order has **no string id**. The API returns the numeric multi-order id; MVRO composes whatever customer-facing label it wants from `original_job_ids` (§4.4).

Submit-time validation (HTTP 400)

Condition	Error
Multi-order options on Fast API	Multi-order merging is supported on Flexible API only
<code>multiorders.radius_m</code> set to any value other than 0	<code>options.multiorders.radius_m</code> is not supported in this version; only exact same-location merging is available

<code>multiorders.force_merge</code> true with <code>multiorders.merge</code> false or absent	<code>options.multiorders.force_merge</code> requires <code>options.multiorders.merge</code>
---	---

A job id inside the reserved multi-order id range	Job ids must be below the reserved multi-order id range
--	--

A re-optimization request whose previous routes reference a multi-order id	Re-optimization must be expressed in original job ids
--	--

A relation referencing a multi-order id	Relations must reference original job ids
--	--

Note what is *not* in this table: nothing about incompatible co-located jobs. Eligibility (§2.4) is a partition, not a validation — a job that cannot merge is simply planned on its own, which is what makes `multiorders.merge: true` safe to set across a mixed workload.

Limitations

1. **No proximity radius.** `multiorders.radius_m` must be 0; only jobs sharing an exact `location_index` are candidates. Customers with per-order geocoding must deduplicate addresses themselves. (*Highlighted in §2.2.*)
2. **Merging is not guaranteed.** It is an optimizer decision, so enabling it does not promise one stop per address, and the same address may be merged in one plan and not in the next as inputs change. Customers who need the guarantee use `multiorders.force_merge` and accept a worse plan (§2.3).
3. **Multi-order ids are per-plan, not per-address.** They must never be stored as durable keys (§2.7).
4. **Time-window intersection is conservative.** Orders with disjoint windows are never candidates, even where a vehicle could have served them in two visits.
5. **Not eligible to merge:** split orders, LIFO-ordered orders, orders with 3D volumes, per-job in-vehicle limits, and per-job depot bindings where the merge would produce both quantities (§2.4).
6. **Shipments are not merged** in v1 — only jobs. A shipment step and a co-located job stay separate steps.
7. **Merging is decided within one plan.** The optimizer weighs each candidate against the plan it is building, not against an exhaustive search over every merge combination; a better set of merges may exist in principle. This is the ordinary heuristic trade-off that applies to every other decision the optimizer makes.
8. **Excluded from simulation mode** ([simulation-mode.md](#)).
9. §2.1 — mixed pickup and delivery on one job — carries none of these limitations. It is unconditional and independent of the multi-order options.

Out of scope (v1)

1. `multiorders.radius_m > 0` — radius-based proximity grouping. The highest-value follow-up, and the clearest remaining gap against RouteQ.

2. **Client-level handover time** — RouteQ's `client_service_duration_s` and `client_id`, a third tier taken at its maximum per client within one address, so several orders for one tenant pay one round of that tenant's paperwork. Worth adding once radius grouping makes large multi-orders common; with `service` and `setup` we already have the two tiers that matter for the cases in front of us.
3. Merging shipment steps, or a shipment step with a job.
4. Merging across locations that are distinct entries in the request, even at the same street address.
5. Portal / NCC UI.

Competitors study

RouteQ — multi-orders ([properties of orders](#)) is the reference implementation and the model this design follows.

Capability	RouteQ	Our v1
Combine orders at one address	Automatic	Opt-in via <code>options.multiorders.merge</code> , because our response introduces new step ids
Merge only where it improves the plan	Default behavior	Same — the default and the recommended mode (§2.3)
Force merging regardless of metrics	<code>force_merge_multiorders</code>	<code>options.multiorders.force_merge</code> — same semantics, same warning
Proximity radius	<code>multiorder_radius_m</code> , default 1 m	<code>options.multiorders.radius_m</code> — not supported, must be 0 ; exact <code>location_index</code> match only
Per-order opt-out	<code>can_be_merged</code> , default true	Same field on the job, same default
Per-order handover, summed	<code>service_duration_s</code>	<code>service</code>
Per-address overhead, maximum	<code>shared_service_duration_s</code>	<code>setup</code>
Per-client handover, maximum per client	<code>client_service_duration_s</code> + <code>client_id</code>	Not in v1
Time windows as a grouping criterion	Yes	Yes — non-empty intersection required for candidacy

Tag / compatibility check when grouping	Yes	Yes — load type and skill rules (§2.4)
Capacity respected when grouping	Yes	Yes — as part of the plan-level decision, not a static rule (§2.4)
Response representation	Not documented	New multi-order id plus <code>original_job_ids</code> (§2.6)

With this revision the only functional gaps against RouteQ are the proximity radius and the client-level handover tier, both listed in §Out of scope.

- **Google Route Optimization API** goes further in a different direction: a shipment holds arrays of pickup and delivery visit requests, so multi-operation stops are declared by the caller rather than discovered by the solver. That is the shape our "future scope" line points at; §2.2 reaches a comparable outcome from the job side without a new top-level object, and additionally decides *whether* the combination is a good idea.
- **Salesforce Field Service and classic VRP solvers** keep the one-operation-per-visit assumption and expect the caller to pre-aggregate — which is exactly where Velocitor is today, with the problem of §1.2 point 3 as the standing hazard.

3. Solution Design (high level)

3.1 How the optimizer decides

Merging is a decision the optimizer makes while planning, not a transformation applied before it. The mechanism has three parts.

Candidates are prepared once. Before planning, the eligibility rules of §2.4 partition each location's jobs into candidate groups, and the aggregation rules of §2.5 define what each candidate would look like as a single stop. Preparing candidates is cheap and deterministic; nothing is committed at this point.

Candidates are screened for feasibility. A candidate that no vehicle in the fleet could serve — its combined load exceeds every compatible vehicle's capacity, or its intersected time window is unreachable — is never offered to the planner, and is instead replaced by the largest sub-groups that *are* servable. This is a cheap static check that removes the most damaging merges before the search ever sees them, and it is what makes the guarantee in §2.3 hold even in the first plan the optimizer constructs.

Merges are formed and dissolved during the search. The optimizer's improvement loop already works by removing parts of the plan and reinserting them differently. Multi-orders join that loop as two symmetric moves: a realized multi-order can be **dissolved** into its component orders, which are then reinserted individually; a set of component orders that a candidate covers can be **formed** into a multi-order and inserted as one stop. Each move is kept only when the total objective improves, by the same acceptance rule that governs every

other move. Over the run, each candidate is therefore tried both ways, and the plan retains whichever it prefers.

The comparison is only meaningful because per-order costs are counted on the original orders (§2.5); with per-stop counting, forming a multi-order would look like a saving regardless of its merits, and the acceptance rule would never dissolve anything.

`multiorders.force_merge` short-circuits all of this: every candidate is formed up front and the dissolve move is disabled.

When the result is written out, each realized multi-order becomes one step carrying its original ids, and orders that were never merged are reported exactly as they are today.

3.2 Why the handover aggregation is decided before planning, not while writing the response

The *decision* to merge belongs in the search, but the *arithmetic* of what a merged stop costs must be settled before the plan is built. It is tempting to leave the orders separate throughout planning and combine them only when writing the response, since the visible artifacts — one step, one summed service time — look like a reporting concern. That does not work:

- **The plan has to be built against the combined handover time.** Two co-located orders of 3 and 2 minutes planned as two visits have the second starting after the first has finished. Merging only at reporting time would stamp the first order's arrival on a step that claims 5 minutes of work, and every later arrival in the route would be wrong by one service block.
- **The intersected time window has to constrain the plan, not describe it afterwards.** Two co-located orders promised 09:00-10:00 and 09:30-11:00 must be driven into 09:30-10:00. A reporting-time merge would let the optimizer place them at 09:05 and 10:45 and then publish a single arrival that honors neither promise.
- **Nothing keeps two co-located orders adjacent unless they are one stop.** The optimizer is free to insert an unrelated visit between them, and often will.

This is also why "merged" and "not merged" are genuinely different states to be compared, rather than two renderings of one plan.

3.3 Re-optimization

A re-optimization request refers to a previous plan, and that plan's response contained multi-order ids. The contract:

- Re-optimization input is always expressed in **original** job ids; multi-order ids are rejected at submit time.
- The previous plan's groupings are recovered from the orders that shared a step, and are re-formed as the starting point for the new plan. A customer's plan therefore stays stable across a re-optimization that changes nothing nearby.
- Where the changes make a previous merge a bad idea — an order removed, a vehicle withdrawn, a window tightened — the same dissolve move that operates in a fresh solve applies here, and the multi-order is broken up.
- Multi-order ids are reassigned for the new plan and may differ from the previous one. This is invisible to any consumer that reads `original_job_ids` instead of caching ids (§2.7).

3.4 Diagnostics

Two questions will dominate support traffic, and the response needs to answer both.

- *"Why were these orders not merged?"* — the honest answer is usually "because the plan was better without it", which is unsatisfying but true. Where a candidate was rejected at the screening stage (§3.1), the more concrete reason is available: no vehicle could carry the combined load. Surface that reason on the orders concerned.
 - *"Why is this multi-order unassigned?"* — an unassigned order is reported under its own id with its own reason, as today. Where several orders of one candidate group are unassigned together, the reason names the group so an operator can tell a genuine single-order failure from a combined-load one. This matters most under `multiorders.force_merge`, where the optimizer was not permitted to avoid the bad merge.
-

4. MVRO Integration Guide

This section exists because §2.7 lets the API return a job id that MVRO never sent, and because that id is not stable across plans. Everything here is MVRO-side work.

4.1 Lift the pickup/delivery validation — ships independently

Remove the "a job may specify pickup or delivery, not both" check from request validation, keeping the three rejections listed in §2.1. Correct the `jobs` section of the API reference, which currently states the restriction as a rule. **This alone closes Velocitor's gap** and needs nothing else from this document.

4.2 Enforce the v1 option restrictions at the edge

Reject at request validation rather than letting the solver decide, so that customers get an actionable message:

- `options.multiorders.radius_m` present and non-zero → 400. **This is the restriction most likely to be hit**, because anyone reading RouteQ's documentation will expect a default of 1 m and may copy a radius across.
- `options.multiorders.force_merge: true` without `options.multiorders.merge: true` → 400.
- An unrecognized key inside `options.multiorders` → 400 rather than silent acceptance, so that a customer who writes RouteQ's flat parameter names inside the object learns immediately instead of getting a plan with merging switched off.

4.3 Treat step ids as opaque and non-durable

Any code path that resolves a step's `id` back to a request job by lookup will miss on a multi-order step. The rule:

if the step has `original_job_ids`:

the step covers those orders, and its id is a per-plan multi-order id

otherwise:

the id is the submitted job id, exactly as today

MVRO must not error, drop the step, or fall back to a null order when the lookup misses. It must also never **persist** a multi-order id as a key — the same address may not be merged next time, or may be merged with a different set of orders. Every stored record keys off the original job ids. Audit at least: response validation, the string-id mapping, webhook payload construction, proof-of-delivery linkage, and any per-order billing counter.

4.4 Fan out to customer orders

The API reports the stop once. MVRO owns the expansion back to per-order records, because only MVRO knows what the customer wants to see:

- **Per-order timing** — all orders in a multi-order share the step's arrival time. If a customer needs per-order completion times, MVRO allocates the step's service duration across `original_job_ids` in submission order, using the per-order service times it already holds from the request.
- **Per-order quantities** — the step's `pickup` and `delivery` are sums. Per-order amounts come from the request, not from the response.
- **Customer-facing id** — a multi-order has no string id. MVRO synthesizes the label, whether that is a joined list of the original ids or the first order plus a "+2 more" marker. The API deliberately does not invent one.

4.5 Round-trip re-optimization in original ids

When MVRO builds a re-optimization request from a previous solution, it must reference the **original** job ids from `original_job_ids`, never the multi-order id. Submitting a multi-order id is a 400. The previous grouping is recovered from the orders that shared a step (§3.3), so passing the original ids is sufficient — MVRO does not need to describe the grouping itself.

4.6 Counting and limits

- **Task limits** are counted on the request, so merging does not change how many tasks a customer may submit.
- **Billing and reporting** must count original orders, not steps. A route with 40 steps may represent 60 orders.
- **unassigned[] needs no special handling** — orders are always reported individually under their own ids (§2.6), so existing per-order failure reporting keeps working.

4.7 Setting expectations with customers

Because merging is a plan-level decision, the SE team should describe the feature as *"the optimizer will combine same-address orders where it helps"*, not *"same-address orders become one stop"*. Two follow-ups to be ready for: a customer comparing two runs and finding the same address merged in one and not the other (expected — the inputs differed), and a customer who needs a hard one-visit-per-address guarantee (that is `multiorders.force_merge`, with the plan-quality warning of §2.3).

4.8 Rollout

The multi-order options default to off, and Velocitor does not need them. Ship §4.1 first and independently; gate §4.2 through §4.7 behind the flags and enable them per account once the fan-out path is verified.

5. Acceptance Walkthrough (worked expectations)

1. **Mixed stop, sufficient capacity:** one job delivering 6 and collecting 4, vehicle capacity 10 → assigned; the vehicle leaves the depot with 6 on board and carries 4 after the stop; route totals show both a delivery of 6 and a pickup of 4. (*Verified on the current staging build.*)
2. **Capacity is checked on the real peak, not the net:** the same job against a capacity-5 vehicle → unassigned for insufficient capacity, even though the net change at the stop is only +2. (*Verified; this is the §1.2 netting problem being caught.*)
3. **Merging off, the default:** two co-located jobs of 3 and 2 minutes → two steps at the same location under their own ids, the second starting after the first finishes. Unchanged from today.
4. **Merging on, and it pays:** the same two small jobs → one step, 5 minutes of service, setup counted once at the larger of the two, `original_job_ids` listing both, and the combined delivery and pickup quantities on the step.
5. **Merging on, and it does not pay:** three co-located orders whose combined load exceeds every vehicle, but which fit comfortably as two vehicles' work → **not merged**; all three are served across two routes as separate steps under their own ids. No order is unassigned. This is the central behavior of §2.3, and the one that distinguishes us from backend-side pre-merging.
6. **Forced merging shows the trade-off:** the same three orders with `options.multiorders.force_merge: true` → merged into one stop that no vehicle can serve, and all three unassigned with a reason naming the group. The comparison between cases 5 and 6 is the customer-facing argument for leaving the flag off.
7. **radius_m rejected:** the same request with `"multiorders": { "merge": true, "radius_m": 1 }` → 400, naming `options.multiorders.radius_m` as unsupported in this version.
8. **Different location entries never merge:** two orders at the same street address registered under different location entries → two steps, even with merging enabled. This is the v1 limitation of §2.2, and the expected support question.
9. **Per-order opt-out honored:** three co-located small orders, one with `can_be_merged: false` → at most a multi-order of two, plus one ordinary job step.
10. **Window intersection:** co-located orders promised 09:00-10:00 and 09:30-11:00 are candidates, and if merged the stop arrives inside 09:30-10:00. Orders promised 09:00-09:30 and 10:00-11:00 are never candidates and always produce two steps.

11. **Eligibility partitions rather than fails:** three orders at one location, one of which requires an extra skill → the extra-skill order is never a candidate; the other two may merge. No error.
 12. **Per-order economics are unbiased:** with `costs.per_order` set, merging two co-located orders does not by itself reduce the plan's cost — the two orders are still counted as two. A merge is chosen for the travel and setup it saves, not for the accounting.
 13. **Unassigned reporting unchanged:** every unassigned order appears under its own id, whether or not it was part of a merge attempt. A multi-order id never appears in `unassigned[ ]`.
 14. **Re-optimization preserves stable groupings:** re-running with nothing changed near a merged address keeps that multi-order intact; removing one of its orders, or the vehicle that served it, lets the remaining orders be re-planned separately.
-

6. Notes

1. **The requirement's "no solver-side merging" line is respected where it counts.** §2.1 — the part Velocitor actually needs — involves no merging at all and is pure API-layer work. §2.2 onward is opt-in and default off. It exists because the merge/don't-merge judgement depends on fleet capacity, which a customer's backend cannot see, and because it closes a named gap against RouteQ.
2. **Why metric-driven rather than merge-everything.** An earlier draft of this document merged every eligible group before planning, which is simpler to build and matches RouteQ's *forced* mode. It was rejected because it reproduces the exact failure of backend-side pre-merging: a combined stop no vehicle can serve, taking every order in it down. Making the merge a decision the optimizer can undo is more work — §3.1 — and it is what makes the feature safe to enable by default on an account.
3. **RouteQ's semantics, our nesting.** The defaults and behavior of `merge`, `force_merge` and `radius_m` are RouteQ's exactly, and `can_be_merged` keeps their name and default outright. What differs is that our three request-level settings live in an `options.multioorders` object rather than at the top of `options`: it keeps the feature's surface in one place, matches how `options.grouping` and `options.objective` are already organized, and gives the §Out of scope follow-ups somewhere to land without new top-level keys. The mapping is spelled out in §2.2 so anyone comparing the two APIs can translate at a glance, and §4.2 rejects RouteQ's flat names rather than ignoring them.
4. **`multioorders.radius_m` is the parameter to watch.** It is shipped only to keep the request shape stable, and `0` is the sole permitted value. Expect it to be the first question from anyone who has read RouteQ's documentation, where the default is 1 m. Radius support is the top item of §Out of scope.
5. **Why a new id and not the smallest original one.** Our one-to-many shipment feature keeps the original pickup id on its group representative, and that precedent was considered and rejected here: a shipment group's representative genuinely *is* the pickup, whereas a multi-order is not any one of its orders. Reusing order 117's id

makes 118 look silently dropped to every consumer that has not been updated. A reserved-range id fails loudly instead, and §4.3 is the rule that handles it.

6. **The id is per-plan, and that is a feature.** Because merging is a decision, a durable "multi-order identity" would be a fiction — the same address legitimately merges differently as inputs change. Making the id obviously ephemeral pushes integrators to key off `original_job_ids`, which is the only stable thing in the response.
7. **`original_job_ids` is worth adding even if merging is never enabled.** It is the extension point for any future feature that renders several request objects as one step, and its presence or absence is a cleaner signal than a type flag.
8. **Documentation.** The API reference needs the `jobs` restriction corrected (§4.1), the four multi-order parameters added with their v1 support status, and `routes[].steps[].original_job_ids` plus the widened step-level `pickup` and `delivery` fields added to the result schema.